

Projet TPI - Share++

Table des matières

1	Analyse préliminaire.....	3
1.1	Rappel du projet.....	3
1.2	Environnement de développement.....	3
1.3	Objectifs.....	4
1.4	Méthode de gestion de projet.....	5
1.5	Planification initiale.....	6
2	Analyse / Conception.....	6
2.1	Concept.....	6
2.2	Stratégie de test.....	7
2.3	Risques techniques.....	8
2.4	Planification.....	9
2.5	Dossier de conception.....	9
3	Réalisation.....	10
3.1	Environnement.....	10
3.1.1	Développement sur laptop :.....	10
3.2	Structure du projet.....	10
3.3	Fonctionnalités techniques importantes.....	11
3.3.1	Workflow Build & Release automatisé.....	11
3.3.2	Logging.....	12
3.3.3	Point d'accès Wi-Fi.....	13
3.3.4	Détection et intégration Tailscale.....	14
3.3.5	Utilisation en ligne de commande (CLI).....	16
3.4	Liste des documents fournis.....	17
4	Conclusions.....	17
5	Annexes.....	18
5.1	Résumé du rapport du TPI / version succincte de la documentation.....	18
5.2	Sources – Bibliographie.....	18
5.3	Journal de travail.....	18
5.4	Manuel d'Installation.....	18
5.5	Manuel d'Utilisation.....	18
5.6	Archives du projet.....	18

1 Analyse préliminaire

1.1 Rappel du projet

Share++ est une application Windows native permettant le partage de fichiers et de répertoires entre les appareils d'un même réseau local. Elle permet de se passer des services de cloud tiers (OneDrive, WeTransfer, Gmail, etc.) pour les transferts internes, garantissant une confidentialité totale des données et une vitesse de transfert maximale limitée uniquement par le débit du réseau local.

Ce TPI reprend le travail réalisé lors du préTPI et y ajoute cinq axes d'amélioration définis dans le cahier des charges, en plus d'apporter une correction des points laissés en suspend du dernier projet :

Connectivité avancée : création d'un point d'accès Wi-Fi (Access Point)

Traçabilité et audit : journalisation persistante des téléchargements

Partage à distance : détection et intégration Tailscale

Expérience utilisateur : refonte de l'interface web de téléchargement

Automatisation et CLI : interface en ligne de commande headless

1.2 Environnement de développement

Le développement a été effectué principalement sur le même laptop Linux personnel (Fedora 43), avec l'ordinateur Windows mis à disposition par le CPNV pour tester l'application et faire la validation finale des fonctionnalités. Le dépôt GitHub du préTPI a été réutilisé après archivage de son état dans une branche dédiée « prétpi ».

Détails de l'environnements :

Catégorie	Technologie / Librairie	Version	Rôle et justification
Langage de programmation	Python 3.13	3.14.3	Langage principal, choisi pour sa simplicité de développement et ses nombreuses librairies
Framework UI	Flet (basé sur Flutter)	0.82.2	Framework de l'interface graphique de l'application, choisi pour son style, sa documentation pleine d'exemples concrets et le développement multiplateforme
Serveur Web	aiohttp	3.13.3	Serveur HTTP asynchrone permettant de gérer les transferts de fichiers sans bloquer l'interface.
Génération QR Code	qrcode, Pillow	8.2 / 11.3.0	Génération dynamique de QR Codes pour faciliter la connexion mobile.
Tests Unitaires	pytest / pytest-asyncio	8.4.2	Tests automatisés pour valider la logique métier.

Packaging	Flet CLI (`flet build`)	0.82.2	Outil de build natif transformant le script Python en exécutable Windows (.exe) avec les dépendances Flutter.
Installeur	Inno Setup	6.7.1	Création d'un assistant d'installation professionnel, répartissant les dépendances entre /Program Files et /Appdata et apportant les fonctionnalités des applications windows au programme.
CI / CD	Workflow Github Actions	-	Automatisation du build et du déploiement des versions (Release) à chaque mise à jour du code.
Licence	MIT	-	Licence permissive permettant l'utilisation, la modification et la distribution libre du logiciel.

1.3 Objectifs

Les objectifs principaux de Share++ sont basé sur les objectifs présents dans le cahier des charges, mais ont été reformulés comme demandé :

- Connectivité Avancée : Création d'un Point d'Accès (Acces Point)
 - Ce module permet de résoudre le problème de la segmentation réseau entre l'hôte et le client mobile. L'hôte créera un réseau wifi temporaire ad-hoc, afin d'intégrer la machine cliente dans le sous-réseau, afin de rendre la page de téléchargement accessible avec le QR Code.
- Traçabilité et Audit : Historique des Téléchargements
 - Couche de supervision du fonctionnement de l'application. Système de journalisation (logging) persistant gardant une trace de chaque requêtes réussies (status 200), ainsi que chaque requête échouées. L'historique affiche le nom du fichier, l'horodatage précis, l'adresse IP du client et tente d'identifier le type d'appareil (User-Agent parsing)
- Partage à Distance : Détection et Intégration Tailscale
 - L'objectif de cette partie est d'étendre la portée du partage au-delà du réseau local physique (LAN) vers un réseau maillé sécurisé (Overlay Network) par le biais d'une détection automatique de la présence du service Tailscale en cours d'exécution sur la machine hôte. Si actif, l'application récupère l'adresse IP « Tailscale » (généralement commençant par 100.x.x.x) et propose de copier un lien sécurisé accessible par tout autre appareil membre du même « Tailnet », permettant le partage de fichiers à travers Internet sans configuration de redirection de port (NAT traversal).
- Expérience Utilisateur : Refonte de l'interface web de téléchargement
 - Le candidat va procéder à une réécriture complète du frontend de la page de téléchargement servie par l'application. L'interface devient « Responsive Design » (adaptée aux mobiles, tablettes et bureaux),

- intègre une charte graphique moderne et offre une prévisualisation plus claire des fichiers disponibles (icônes selon le type, taille, date).
- Automatisation et CLI : Interface en ligne de commande
 - Ouverture de l'application aux utilisateurs avancés et aux scripts d'automatisation sera proposée.
 - Pour se faire, il procédera au développement d'un exécutable en ligne de commande (sharepp.exe) permettant de lancer un serveur headless (sans interface graphique) sur un répertoire spécifique.
 - Syntaxe cible : Sharepp /chemin/vers/dossier – port 3001 – secure
 - Fonctionnalités : le paramètre –secure active une protection par mot de passe (Basif Auth ou page de garde), et le terminal affiche un QR Code généré en ASCII/UTF-8 ainsi que les logs de connexion en temps réel.

Les corrections des défauts présents non réglés du préTPI se feront une fois ces fonctionnalités clés réalisées.

Le serveur génère une page web simple listant les fichiers disponibles, avec des liens de téléchargement directs. L'application est compilée en exécutable autonome, ne nécessitant aucune installation de Python, ou d'aucune autre dépendance sur la machine cible.

1.4 Méthode de gestion de projet

La méthode Scrum a été choisie car elle permet une livraison incrémentale adaptée à un projet de courte durée avec des objectifs bien définis.

1.5 Planification initiale

Le projet débute le 23 Avril 2026 et dure jusqu'au 20 Mai 2026. Il est découpé en 4 sprints :

1. Sprint 1 (connectivité, point d'accès, traçabilité)
2. Sprint 2 (Partage à distance et utilisation CLI)
3. Sprint 3 (Expérience utilisateur)
4. Sprint 4 (Correction préTPI et derniers correctifs)

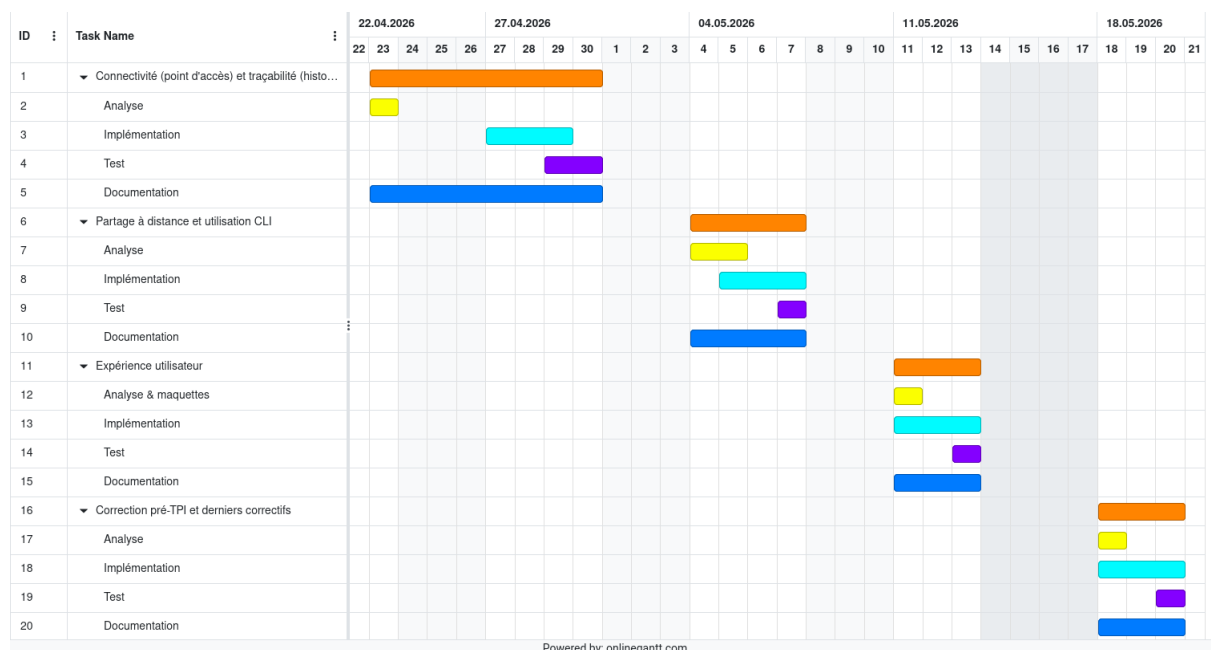


Figure 1: Gantt du planning initial

2 Analyse / Conception

2.1 Concept

Le concept est repris du projet préTPI (Share++ v1.0) et a été étendu pour inclure les nouveaux objectifs.

Le projet repose toujours sur une architecture client-serveur gérée par une application Windows native. Le code suit également toujours le modèle MVC afin de préserver la bonne organisation du code et une future maintenance de l'application :

- **Backend** : Un serveur aiohttp asynchrone gère les requêtes HTTP et le service des fichiers. Le backend sera étendu afin de supporter l'exposition de deux nouvelles interfaces réseau. La première interface est Tailscale
- **Frontend** : Une page html listant les fichiers partagés servie lorsque le partage est actif
- **Persistence** : Les paramètres de l'utilisateur, ainsi que des logs sont persistents et stockés dans des fichiers .json ou .log dédiés.
- **Application** : Une application windows native servant à gérer le serveur et ses

paramètres.

Analyse fonctionnelle :

Le cas d'utilisation principal reste inchangé : l'utilisateur sélectionne un dossier local, démarre un serveur de fichier, et les autres appareils y accèdent via un QR code ou une URL afin de télécharger les fichiers partagés. Les nouveaux cas d'utilisation sont les suivants :

- Un utilisateur souhaitant partager des fichiers avec un appareil mobile sans être sur le même réseau peut activer un point d'accès Wi-Fi ad-hoc depuis l'application, ce qui crée un sous-réseau dédié entre l'hôte et le client et permet ainsi d'accéder au partage plus facilement.
- Un utilisateur souhaitant partager des fichiers à distance (hors LAN) et disposant de Tailscale peut utiliser l'adresse Tailscale générée automatiquement par l'application.
- Un utilisateur avancé ou souhaitant automatiser le partage peut utiliser l'interface CLI pour démarrer un serveur headless sans interface graphique.
- L'historique des connexions est conservé dans un fichier de log persistant, consultable après la fermeture de l'application, avec le type d'appareil identifié via le User-Agent.

J'ai décidé de garder la licence MIT pour ce projet, car il s'agit de la licence la plus permissive au niveau de la transparence du code et sa réutilisation pour autrui.

Voici la maquette de l'application

2.2 Stratégie de test

4 types de tests sont

La stratégie de test vise à valider les nouvelles fonctionnalités tout en s'assurant que les fonctionnalités existantes n'aient pas régressé.

Types de tests :

- 1. Tests unitaires** : Vérification de la logique de parsing du User-Agent, de l'écriture et de la rotation des logs dans le fichier `sharepp.log`, et de la détection de Tailscale (présence du service, récupération de l'adresse IP).
- 2. Tests d'intégration** : Vérification que le serveur `aiohttp` expose correctement les fichiers sur les différentes interfaces réseau (LAN, hotspot ad-hoc, Tailscale), et que les logs sont correctement écrits lors des requêtes HTTP.
- 3. Tests End-to-End** : Scan du QR code depuis un smartphone connecté au point d'accès ad-hoc créé par l'application, téléchargement de fichiers et de répertoires, et vérification de la traçabilité dans le fichier log.
- 4. Tests CLI** : Vérification que le serveur headless se lance correctement avec les paramètres attendus (`--port`, `--secure`), et que le QR code ASCII s'affiche dans le terminal.

5. Tests de non-régression : Exécution de l'ensemble des tests unitaires et d'intégration du préTPI afin de garantir qu'aucune fonctionnalité existante n'a été cassée par les modifications apportées.

Moyens de mise en œuvre :

Les tests unitaires et d'intégration sont automatisés avec pytest, en continuité du projet préTPI. Les tests End-to-End sont réalisés manuellement avec le même matériel qu'au préTPI (laptop Lenovo Yoga sous Fedora 43 (Linux) comme serveur, smartphone Android Realme Neo7 comme client).

Couverture :

Les tests ne sont pas exhaustifs. Ils couvrent les fonctionnalités principales et les points de sécurité. Les données de test reprennent celles du préTPI auxquelles s'ajoutent des tests spécifiques pour les fonctionnalités de CLI et de logging.

2.3 Risques techniques

	Risque	Probabilité	Impact	Criticité	Solution
1	La création d'un point d'accès Wi-Fi ad-hoc avec python ne semble pas très courant, la fonctionnalité pourrait être fragile et rendre l'application instable	3	3	9	Utiliser une solution bien documentée
2	Régression après l'ajout de fonctionnalités avancées	1	4	4	Tester l'application après chaque release afin de s'assurer au fur et a mesure que rien ne régresse.
3	La refonte de l'interface web pourrait introduire des régressions sur la logique de téléchargement de fichiers et de répertoires.	1	4	4	Maintenir les tests d'intégration existants et les exécuter systématiquement après chaque modification du frontend.
4	L'application est trop dure à utiliser pour un utilisateur final peu	3	4	12	Refaire des maquettes et apporter des changements de design

	expérimenté				de l'application et de la page de téléchargement pour que l'expérience soit ergonomique
5	L'utilisation des fonctions actuels pour le CLI pourrait ne pas fonctionner car elles dépendent de l'interface graphique	4	4	16	Apporter des modifications aux fonctions actuels du programme afin de les rendre compatible avec l'utilisation par lignes de commandes

2.4 Planification

Révision de la planification initiale du projet :

- *planning indiquant les dates de début et de fin du projet ainsi que le découpage connu des diverses phases.*
- *partage des tâches en cas de travail à plusieurs.*

*Il s'agit en principe de la planification **définitive du projet**. Elle peut être ensuite affinée (découpage des tâches). Si les délais doivent être ensuite modifiés, le responsable de projet doit être avisé, et les raisons doivent être expliquées dans l'historique.*

2.5 Dossier de conception

Fournir tous les document de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*
- *site web: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, ...*
- *bases de données: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.*
- *programmation et scripts: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme...*

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

3 Réalisation

3.1 Environnement

J'ai utilisé le même environnement pour ce travail que pour mon projet préTPI, c'est à dire que j'ai utilisé un laptop Linux personnel pour le développement, puis j'ai testé l'application depuis l'ordinateur sous Windows 11 mis à notre disposition par le CPNV. Tester l'application sur un poste Windows est important comme je développe sur un environnement techniquement non supporté par mon application (application windows développée sur un poste Linux). Il est donc possible de valider ainsi les fonctionnalités implémentées.

J'ai repris le dépôt Github du projet préTPI (<https://github.com/NathanFilipowicz/Sharepp>) et ai continué de l'utiliser pour ce projet, après avoir archivé l'état du dépôt dans une branche 'préTPI' dédiée. J'ai également créé des répertoires afin de ne pas garder tous les documents dans un seul répertoire /docs comme lors de mon dernier projet.

3.1.1 Développement sur laptop :

J'ai principalement utilisé mon laptop personnel pour réaliser ce travail. J'ai conservé la logique de séparation de code de mon projet préTPI. Les fichiers de code suivent la logique MVC (Model-View-Contrôleur) traduite ainsi dans mon projet :

- /src/controllers, code dédié à la logique métier de l'application. Les fonctionnalités, l'orchestration
- /src/models, code dédié à la persistance des données
- /src/views, code dédié à l'interface graphique de l'application et de la page de téléchargement

ainsi qu'un main.py servant de point d'entrée de l'application flet, et détecte désormais le niveau de privilège afin de pouvoir s'élever afin que la fonctionnalité de création d'access point fonctionne

Développement sur ordinateur CPNV :

J'ai eu besoin de migrer le projet sur l'ordinateur fixe du CPNV afin de pouvoir intégrer la fonctionnalité spécifique windows du bouton contextuel de l'explorateur de fichiers. Pour ceci, j'ai cloné le projet depuis mon dépôt github afin de pouvoir travailler sur l'environnement et pouvoir tester l'application directement. J'ai aussi utilisé cet environnement pour créer l'installateur InnoSetup pour le build final windows de l'application.

3.2 Structure du projet

La structure est également représentée plus succinctement dans le README du projet. L'architecture choisie est l'architecture MVC (Model, View, Controller) me permettant de séparer la logique de mon code.

```
Sharepp/
├── .github/
│   └── workflows/
│       └── build-release.yml
├── docs/
│   ├── Dossier_de_projet.odt
│   ├── Journal_de_travail.ods
│   ├── Journal_de_test.pdf
│   └── maquette_app.png
```



3.3 Fonctionnalités techniques importantes

3.3.1 Workflow Build & Release automatisé

Un workflow GitHub Actions (.github/workflows/build-release.yml) automatise entièrement le build windows de l'application, ainsi que le packaging et la publication de l'installateur Windows sous forme de release versionnée sur le dépôt Github. Il se déclenche à chaque push s'accompagnant d'un tag de version -v*.

Etapes du workflow

1. Récupération du code du dépôt et configuration de Python 3.14.3.
2. Extraction de la version depuis le tag Git (v1.0.0 → 1.0.0).
3. Installation des dépendances Python depuis requirements.txt.
4. Installation du SDK Flutter.
5. Build de l'exécutable Windows via flet build windows.
6. Installation d'InnoSetup avec Chocolatey.
7. Mise à jour dynamique du numéro de version dans setup.iss.
8. Génération de l'installateur .exe via **ISCC.exe** (Inno Setup Command-Line Compiler).
9. Création de la release GitHub et upload de l'installateur.

Ce pipeline garantit la reproductibilité des releases, leur versionnement et permet aux utilisateurs de pouvoir facilement et rapidement accéder à une version à jour et facilement installable de l'application.

3.3.2 Logging

- Utilisation de la librairie « logging » de la librairie standard de python.
- Écriture dans un fichier `sharepp.log` avec rotation automatique pour éviter les fichiers logs trop volumineux.
- Ajout d'une nouvelle donnée dans les logs : Le type d'appareil utilisé pour accéder au partage (récupéré grâce au User-Agent parsing).
- Intégration dans le système précédent de logs sans trop de réécriture

L'application disposait déjà d'un affichage des logs en temps réel dans son interface dès sa version 1.0 (préTPI), mais ces informations étaient perdues à chaque fermeture. Pour répondre à l'objectif de traçabilité du cahier des charges, un système de journalisation persistant a été ajouté au modèle, permettant de conserver un historique des logs précédemment affichés qu'en temps réel. Ceci permet à l'utilisateur de pouvoir consulter les logs a posteriori, sans avoir à ouvrir l'application. L'utilisateur aura également droit à d'avantage d'informations sur le type d'appareil (« [Mobile[Android]] », « [PC[Windows]] »...) qui s'est connecté à son serveur.

La librairie logging de python repose sur trois concepts : le **Logger**, qui reçoit les messages et filtre par niveau de sévérité ; le **Handler**, qui dirige les messages vers leur destination (ici un fichier `sharepp.log`), et le **Formatter**, qui définit la mise en forme de chaque entrée.

```
# Persistent log file path
self.log_path = self.config_dir / "sharepp.log"
self._setup_file_logger()

def _setup_file_logger(self):
    # Identify log file (create if not exists)
    self.file_logger = logging.getLogger("sharepp")
    # Accept logs of all severity levels (DEBUG == lowest priority)
    self.file_logger.setLevel(logging.DEBUG)

    if not self.file_logger.handlers:
        handler = RotatingFileHandler(
            self.log_path, maxBytes=1_000_000, backupCount=3, encoding="utf-8"
        )
        formatter = logging.Formatter(
            "%(asctime)s [%(levelname)s] %(message)s",
            datefmt="%Y-%m-%d %H:%M:%S"
        )
        handler.setFormatter(formatter)
        self.file_logger.addHandler(handler)
```

Emplacement du fichier de log

Le fichier logs est enregistré dans le répertoire Appdata de Windows, aux côtés de settings.json, ce répertoire est le répertoire conseillé pour les documents dont il faut le droit d'accès en écriture.

Retrouver le fichier :

%APPDATA% -> Roaming\SharePlusPlus\sharepp.log

Format des logs

2026-04-27 12:06:43 [INFO] Serveur ouvert sur le port 8080

2026-04-27 12:06:54 [INFO] Connexion de 10.227.140.6 [PC (Linux)]

2026-04-27 12:06:54 [WARNING] 10.227.140.6 [PC (Linux)] télécharge favicon.ico

2026-04-27 12:07:01 [ERROR] Serveur fermé

Rotation automatique

Pour ne pas réécrire continuellement sur le même fichier log, un RotatingFileHandler a été utilisé, servant à créer un nouveau fichier log après que celui-ci ait dépassé 1 Mo, et ne conserve que trois versions de fichiers logs. Comme toutes les logs affichés sur l'application sont enregistrés automatiquement, cela garantit que l'espace disque autorisé pour les logs soit limité à 4 Mo.

Intégration avec le contrôleur

La méthode Controller.log(message, niveau) appelle désormais model.log_to_file() pour la persistance du log, et continue de l'afficher dans l'interface Flet en même temps.

Identification des appareils (User-Agent Parsing)

Chaque entrée de log issue d'une connexion entrante, inclut une identification de l'appareil client déduite par le header HTTP User-Agent. La méthode parse_user_agent() du contrôleur récupère le type d'appareil (Mobile, Tablette, PC) et le système d'exploitation (Android, iOS, Windows, Linux, macOS) de chaque requêtes de téléchargement entrante (pas pour les logs informatifs de type Password protection enabled).

Exemples de sorties :

Mobile (Android)

PC (Windows)

Appareil inconnu

3.3.3 Point d'accès Wi-Fi

Le cas d'usage principal de Share++ consiste à partager des fichiers entre un ordinateur de bureau et un smartphone. Cependant, il est commun que le smartphone et l'ordinateur ne partagent pas le même sous-réseau, si le smartphone est connecté en 5g par exemple. Dans ce contexte, le client mobile n'est pas en mesure d'accéder à la page de téléchargement générée par le serveur aiohttp.

La solution retenue est de permettre à l'hôte de créer lui-même un réseau Wi-Fi ad hoc temporaire. Le smartphone se connecte à ce réseau, rejoignant ainsi le sous-réseau de l'hôte et rendant la page accessible.

Implémentation

La logique de création, démarrage et arrêt du point d'accès est inscrite dans le fichier `src/controllers/hotspot_controller.py`, afin de respecter l'architecture MVC du projet.

Fonctions ajoutées

- `get_hotspot_status()` : lance la commande ``netsh wlan show hostednetwork`` et détecte si le point d'accès est actif, en traitant la sortie de la commande en français et en anglais (bilingue, car le retour de la commande sur la version francophone de windows est en français).
- `setup_and_start(ssid, password)` : valide le mot de passe (minimum 8 caractères, exigence de tout mot de passe Wi-Fi WPA2), exécute la commande de configuration puis démarre le réseau.
- `stop_hotspot()` : arrête le point d'accès.
- `execute_command(command)` : wrapper autour de `subprocess.run` avec le flag `CREATE_NO_WINDOW (0x08000000)` pour éviter les flashes de terminal Windows. Implémenté avec l'aide de Gemini (voir journal de travail).
- `ensure_admin()` : vérifie les droits administrateur et relance l'application avec élévation si nécessaire.

Intégration dans l'interface

La hotspot card indépendante présente dans une version antérieure a été supprimée pour une intégration dans la card principale de configuration du partage. Deux contrôles ont été ajoutés :

Un bouton "Créer un hotspot Wi-Fi" permet d'activer la fonctionnalité de manière persistante dans `settings.json`. Ce bouton est affiché sur tous les systèmes d'exploitation mais est désactivé sur les systèmes non-Windows.

Un bouton "Configurer le réseau Wi-Fi" ouvre un `AlertDialog` contenant les champs SSID et mot de passe. Conformément à l'architecture MVC, la création et l'affichage de ce dialog sont entièrement gérés dans `view.py` le contrôleur se limite à valider et persister les données via `save_hotspot_credentials()`.

Flux de l'expérience utilisateur

Lorsque l'utilisateur active le hotspot depuis l'interface :

1. Share++ crée le réseau avec le SSID et le mot de passe configurés.
2. Une bannière d'instruction apparaît au-dessus des tuiles de connexion, invitant l'utilisateur à se connecter d'abord au réseau Wi-Fi avant de scanner l'URL.
3. Un QR code Wi-Fi est disponible dans une `ExpansionTile` dédiée, encodant la chaîne `WIFI:T:WPA;S:{ssid};P:{pwd};H;;` qui permet de se connecter automatiquement en le scannant depuis un smartphone.
4. L'adresse IPv4 de la machine servant le serveur de fichier : `http://192.168.137.1:{port}` apparaît comme option de connexion supplémentaire avec un QR code de connexion aux côtés des adresses de l'ip locale et de l'ip Tailscale.
5. Une fois connecté au hotspot, l'utilisateur peut scanner le QR code issu du hotspot adhoc pour accéder aux fichiers. A l'arrêt du serveur de fichier, le réseau adhoc se ferme automatiquement.

Problèmes rencontrés

L'utilisation de netsh (interpréteur de commandes réseau Windows) pour la création de l'accès point oblige à tester la fonctionnalité sur une machine Windows. Le problème étant que l'ordinateur de test mis à ma disposition pour ce travail n'avait pas de carte Wi-Fi et ne pouvait donc pas afficher les commandes netsh recherchées. J'ai donc dû me procurer une clé USB Wi-Fi afin de pouvoir tester et corriger l'implémentation.

Même après avoir obtenu la clé Wi-Fi TP-Link et installé les drivers, je me suis rendu compte que la clé ne supportait pas la fonctionnalité 'réseau hébergé' (netsh wlan show drivers → Réseau hébergé pris en charge : non), prérequis nécessaire à la commande netsh wlan hostednetwork.

Le Mercredi 06 mai, M. Wuliamoz et moi avons testé plusieurs adaptateurs USB ainsi qu'un laptop windows, jusqu'à ce que nous ayons trouvé un adaptateur USB qui supporte la fonctionnalité me permettant alors de poursuivre l'implémentation avec un peu de retard.

3.3.4 Détection et intégration Tailscale

Contexte et objectif

Share++ est conçu pour fonctionner sur un réseau local. Cependant, il est fréquent qu'un utilisateur souhaite partager des fichiers avec un appareil situé hors de son réseau (comme avec un smartphone en 5g par exemple), sans pour autant avoir configuré manuellement une redirection de port sur son routeur. Tailscale est un service de réseau maillé sécurisé basé sur WireGuard qui résout exactement ce problème : chaque appareil du sous-réseau Tailscale appelé "Tailnet" reçoit une adresse IP fixe de plage 100.x.x.x et peut communiquer avec les autres machines du Tailnet, pourvu qu'ils aient un accès à internet.

L'objectif est de détecter automatiquement la présence du service Tailscale sur la machine hôte au moment du démarrage du serveur, et de proposer un lien de téléchargement supplémentaire via cette interface, sans aucune configuration manuelle de la part de l'utilisateur.

Architecture network_controller.py

La logique de détection réseau a été écrite dans un fichier dédié : src/controllers/network_controller.py, conformément à l'architecture MVC du projet. Ce contrôleur expose trois fonctions publiques :
get_local_ip() : détecte l'adresse IP du réseau local (précédemment directement présent dans app_controller.py).
get_tailscale_ip(port) : détecte l'adresse IP Tailscale et valide son accessibilité.
get_hotspot_ip() : détecte l'adresse IP de l'interface du point d'accès Wi-Fi ad hoc (voir section 3.3.3).

Détection de l'adresse IP Tailscale

Pour détecter la présence de Tailscale sur la machine de l'utilisateur, on lance une commande tailscale CLI permettant d'afficher l'ip Tailscale de la machine.

La détection se déroule en deux étapes. D'abord, tailscale status est exécuté via subprocess.run() avec un timeout de 3 secondes. Son code de retour détermine si le service est actif. Si le service est inactif, aucun changement dans l'application ne sera observé.

Si le service est actif, tailscale ip --4 est exécuté et récupère l'adresse IPv4 de Tailscale. L'adresse est aussi validée par une expression régulière garantissant qu'elle appartient bien à la plage Tailscale.

Validation de l'adresse IP Tailscale

Pour s'assurer que la connexion à la page de téléchargement fonctionne correctement via l'IP Tailscale, une méthode privée _is_reachable() a été implémentée, lançant une tentative de connexion via la librairie socket afin de pouvoir proposer la fonctionnalité à l'utilisateur, que si elle fonctionne vraiment.

```
def _is_reachable(ip, port, timeout: float = 3.0):
    try:
        with socket.create_connection((ip, port), timeout=timeout):
            return True
    except OSError:
        return False

def _detect_tailscale_ip():
    # Is tailscale active: "tailscale status" output
    try:
        status = subprocess.run(
            ["tailscale", "status"],
            capture_output=True, text=True, timeout=3
        )
        if status.returncode != 0:
            return None

        # get tailscale ip for this machine in IPv4
        result = subprocess.run(
            ["tailscale", "ip", "--4"],
            capture_output=True, text=True, timeout=3
        )
        ip = result.stdout.strip()
        if ip and re.match(r"^100\.([7-9]\d|1[0-1]\d|12[0-7])\.\d{1,3}\.\d{1,3}$", ip):
            return ip

    except Exception as e:
        print(f"Error during Tailscale ip detection: {e}")

    return None
```

Alternative envisagée

En connaissant la plage standard d'attribution d'adresses IPV4 de Tailscale, il aurait été possible de récupérer les adresses IPV4 de la machine via la librairie python standard 'socket'. Cela aurait eu l'avantage de fonctionner en l'absence de tailscale dans le PATH. Cette méthode n'a pas été retenue car une adresse pourrait commencer par 100. sans venir de Tailscale, rendant la détection moins fiable. La méthode CLI ayant fonctionné directement, cette alternative n'a pas été poursuivie.

Validation de l'accessibilité

La fonction `get_tailscale_ip(port)` valide l'adresse IPV4 Tailscale détectée via une tentative de connexion TCP (`socket.create_connection`) sur le port du serveur. Cela sert à valider le bon fonctionnement du lien de téléchargement hors-site avant de le proposer à l'utilisateur.

`get_local_ip()`

La fonction `get_local_ip()` détecte l'IP locale via une connexion UDP factice vers 8.8.8.8. L'implémentation reprend du code directement de la documentation officielle disponible dans la section sources de ce document.

Intégration dans le flux de démarrage du serveur

La détection est déclenchée automatiquement à chaque démarrage du serveur de fichier, dans la méthode `start_server()` du contrôleur. Elle s'exécute dans un thread séparé via `asyncio.to_thread()` afin de ne pas bloquer la boucle d'événements Flet pendant le scan réseau (`subprocess.run()` est bloquant) :

```
local_ip, tailscale_ip = await asyncio.to_thread(self._detect_all_ips)
await self._show_connection_info(local_ip, tailscale_ip)
```

La méthode `_show_connection_info()` génère ensuite les QR codes et met à jour l'interface selon les IPs disponibles :

Si seul le LAN est disponible : un seul ExpansionTile QR est affiché (comportement inchangé par rapport à la version préTPI).

Si Tailscale est détecté : deux ExpansionTiles indépendants sont affichés, un pour l'adresse LAN et un pour l'adresse Tailscale.

Aucune donnée liée à Tailscale n'est inscrite dans le fichier `settings.json`. La détection est donc entièrement en mémoire et se relance à chaque session afin de rendre l'expérience utilisateur plus fiable.

Interface utilisateur

L'affichage des QR codes a alors été refait pour épurer l'interface utilisateur. Les QR codes ne s'affichent plus en grand au démarrage du serveur mais sont masqués par défaut dans des composants `ft.ExpansionTile`. Chaque tile expose sur lui de manière toujours visible :

L'URL du partage, sélectionnable.

Un bouton d'ouverture dans le navigateur.

Un bouton de copie dans le presse-papier.

Le bouton d'arrêt du serveur reste toujours visible au-dessus des ExpansionTiles

3.3.5 Utilisation en ligne de commande (CLI)

Contexte et objectif

Share++ est conçu comme une application graphique Windows. Cependant, certains power users apprécieront d'avoir la possibilité d'ouvrir un serveur de fichier sécurisé en une ligne de commande directement dans leur terminal. L'objectif de cette fonctionnalité de permettre à l'utilisateur d'utiliser l'**outil** Share++ sans passer par

son interface graphique, en préservant les fonctionnalités principales telle que le qr code, la protection par mot de passe, les logs.

Méthode retenue

L'exécutable principal de l'application détecte l'argument `--nogui` et se lance dans le terminal, sans jamais ouvrir de fenêtre de l'application. Un chemin en tant qu'argument obligatoire suivi d'arguments optionnels : `--secure --password --`

Analyse des variantes

Deux approches principales ont été envisagées pour l'implémentation de cette fonctionnalité :

- Variante 1 Exécutable unique, deux modes : `main.py` détecte la présence du flag `--nogui` au démarrage et délègue l'exécution à un module CLI dédié, sans jamais initialiser Flet.

Variante B — Exécutable séparé : un second exécutable `sharepp-cli.exe` est buildé indépendamment, sans aucune dépendance Flet, produisant un binaire plus léger.

La variante A a été retenue pour les raisons suivantes : elle maintient un point d'entrée unique pour les utilisateurs, simplifie le pipeline de build et de distribution (un seul installateur `InnoSetup`), et évite la maintenance de deux artefacts distincts. La légèreté potentielle de la variante B n'est pas un argument décisif dans ce contexte, l'exécutable étant déjà distribué avec l'intégralité des dépendances Python.

3.4 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- *le rapport de projet*
- *le manuel d'Installation (en annexe)*
- *le manuel d'Utilisation avec des exemples graphiques (en annexe)*
- *autres...*

4 Conclusions

Développez en tous cas les points suivants:

- *Objectifs atteints / non-atteints*
- *Points positifs / négatifs*
- *Difficultés particulières*
- *Suites possibles pour le projet (évolutions & améliorations)*

5 Annexes

5.1 Résumé du rapport du TPI / version succincte de la documentation

5.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, date, titre, auteur)... Et de toutes les aides externes (noms)

5.3 Journal de travail

Date	Durée	Activité	Remarques

5.4 Manuel d'Installation

5.5 Manuel d'Utilisation

5.6 Archives du projet

Media, ... dans une fourre en plastique